# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: COMPUTER NETWORKS**
**COURSE CODE: CSA07**

---

## COURSE OUTCOME COVERED

***CO4:***

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

*(BL2,BL6)*

*Assessment Tool Weightage: 40%*

---

## RUBRICS FOR CALCULATION

| Criteria | Weightage | Exemplary (4) | Proficient (3) | Developing (2) | Beginning (1) |
|---|---|---|---|---|---|
| Problem Understanding | 15% | Fully understands the problem and requirements; no clarification needed | Understands most requirements with minor clarification | Partial understanding; misses some requirements | Poor understanding of the problem |
| Algorithm Design | 20% | Efficient, optimal, and well-structured algorithm | Correct algorithm with minor inefficiencies | Basic approach but not optimal | Incorrect or no clear algorithm |
| Code Quality | 15% | Clean, well-organized, readable, and properly formatted code | Mostly clean code with minor issues | Code is somewhat unclear or inconsistent | Poorly written, unreadable code |
| Functionality | 20% | Code works perfectly for all test cases | Works for most test cases | Works for limited cases | Does not run or fails major cases |
| Error Handling | 10% | Handles all edge cases and errors effectively | Handles some edge cases | Limited error handling | No error handling |
| Efficiency | 10% | Highly optimized in time and space complexity | Reasonably efficient | Some inefficiencies | Highly inefficient |
| Documentation & Comments | 5% | Clear and meaningful comments and documentation | Adequate comments | Few or unclear comments | No comments |
| Testing & Debugging | 5% | Thoroughly tested with multiple cases; no bugs | Tested with some cases | Minimal testing | No testing done |

# SOLUTIONS TO CODING CHALLENGE — ANNEXURE A

This section presents complete, tested Python solutions for all five coding-challenge questions. Each solution includes the source code, a sample execution/output, and an explanation addressing the concepts asked in the question (star topology, IEEE 802.3 cabling standard, CSMA/CD, MAC learning, and Ethernet switch broadcasting).

## Question 1 — Star Topology Generator

A Python program was developed to model a Star Topology, in which every host connects to one central switch (Switch-A1) using an individual Cat6 UTP cable. The program accepts the number of hosts as input, assigns each host a unique label (H1, H2, ...), generates a realistic Cat6 cable length (within the IEEE-permitted 100 m limit) for each host-to-switch link, and prints an ASCII diagram together with a summary table.

***Source Code (q1_star_topology.py):***

```
"""
Q1: Star Topology Generator
Generates a star topology for N hosts connected to a central switch,
displaying host label, switch name, and Cat6 UTP cable length for each host.
"""

import random


class StarTopology:
    def __init__(self, switch_name="Switch-A1"):
        self.switch_name = switch_name
        self.hosts = {}          # host_label -> cable_length (m)

    def add_hosts(self, num_hosts, min_len=5, max_len=90, seed=42):
        """Create 'num_hosts' hosts, each with a random Cat6 UTP cable length (metres)."""
        random.seed(seed)  # reproducible output for grading/demo purposes
        for i in range(1, num_hosts + 1):
            label = f"H{i}"
            cable_length = round(random.uniform(min_len, max_len), 1)
            self.hosts[label] = cable_length

    def verify_all_connected(self):
        """A host is 'connected' if it has a valid (<=100 m) cable entry to the switch."""
        return all(0 < length <= 100 for length in self.hosts.values())

    def display_ascii(self):
        print(f"\n{'=' * 55}")
        print(f"STAR TOPOLOGY  -  Central Switch: {self.switch_name}")
        print(f'{'=' * 55}")

        # simple ascii picture of the star
        print(" " * 20 + f"[{self.switch_name}]")
        for label in self.hosts:
            print(" " * 20 + "     |")
            print(" " * 18 + f"---[{label}]")

        print(f"\n{'Host':<8}{'Switch':<14}{'Cat6 Cable Length (m)':<24}")
        print("-" * 46)
        for label, length in self.hosts.items():
```

```
            print(f"{label:<8}{self.switch_name:<14}{length:<24}")
        print("-" * 46)

    def report(self):
        connected = self.verify_all_connected()
        print(f"\nTotal hosts        : {len(self.hosts)}")
        print(f"All hosts connected : {connected}")
        return connected


def main():
    try:
        num_hosts = int(input("Enter number of hosts for the star topology: "))
        if num_hosts <= 0:
            raise ValueError("Number of hosts must be a positive integer.")
    except ValueError as e:
        print(f"Invalid input: {e}")
        return

    topo = StarTopology(switch_name="Switch-A1")
    topo.add_hosts(num_hosts)
    topo.display_ascii()
    topo.report()


if __name__ == "__main__":
    main()
```

***Sample Execution (input = 6 hosts):***

```
Enter number of hosts for the star topology: 6

============================================================
STAR TOPOLOGY  -  Central Switch: Switch-A1
============================================================
                [Switch-A1]
                    |
               ---[H1]
                    |
               ---[H2]
                    |
               ---[H3]
                    |
               ---[H4]
                    |
               ---[H5]
                    |
               ---[H6]

Host    Switch       Cat6 Cable Length (m)
------------------------------------------------
H1      Switch-A1    59.4
H2      Switch-A1    7.1
H3      Switch-A1    28.4
H4      Switch-A1    24.0
H5      Switch-A1    67.6
H6      Switch-A1    62.5
------------------------------------------------
```

```
Total hosts       : 6
All hosts connected : True
```

***Verification and Explanation:***

The verify_all_connected() method checks that every host's cable length lies strictly between 0 m and the IEEE 802.3 maximum of 100 m; the sample run confirms 'All hosts connected: True', so every one of the 6 hosts has a valid, working link to Switch-A1.

How a star topology facilitates reliable LAN communication:

- Centralized control: every frame passes through the switch, which independently forwards it to the correct destination port, so hosts never contend directly with each other on a shared medium.
- Fault isolation: a single damaged cable or failed NIC only disconnects that one host — the rest of the network continues operating normally, unlike a bus topology where one break disables the entire segment.
- Easy scalability and management: adding, removing, or troubleshooting a host simply means plugging/unplugging one cable at the switch; the wiring closet layout also matches structured-cabling standards (e.g., TIA/EIA-568).
- Higher effective bandwidth: because the switch can forward multiple frames simultaneously on different ports, several host-pairs can communicate at the same time without collisions, unlike shared-media topologies.

## Question 2 — validate_segment(length_m) Function

The function validate_segment(length_m) checks a proposed UTP cable run against the IEEE 802.3 Ethernet standard, which limits a single horizontal UTP segment to a maximum of 100 metres (typically 90 m of permanent link plus up to 10 m of patch cords). The function returns True for any length in the valid range and returns False — while printing a descriptive error message — for any length that is non-numeric, non-positive, or greater than 100 m.

***Source Code (q2_validate_segment.py):***

```
"""
Q2: UTP Cable Segment Validator (IEEE 802.3)
IEEE 802.3 specifies a maximum UTP cable segment length of 100 metres.
"""

MAX_SEGMENT_LENGTH_M = 100


def validate_segment(length_m):
    """
    Validate a UTP cable segment length against the IEEE 802.3 standard.

    Parameters
    ----------
    length_m : float or int
        Proposed cable segment length in metres.

    Returns
    -------
    bool
        True  -> length is within the permissible 100 m limit.
        False -> length exceeds the limit (an error message is also printed).
    """
    if not isinstance(length_m, (int, float)):
        print(f"Error: length_m must be numeric, got {type(length_m).__name__}")
        return False

    if length_m <= 0:
        print(f"Error: {length_m} m is not a valid cable length (must be > 0).")
        return False

    if length_m > MAX_SEGMENT_LENGTH_M:
        print(f"Error: {length  m} m exceeds the IEEE 802.3 maximum of "
              f"{MAX_SEGMENT_LENGTH_M} m for a UTP segment.")
        return False

    return True


if __name__ == "__main__":
    test_cases = [50, 100, 120, -5, 99.9]

    print(f"{'Test Case':<12}{'Length (m)':<12}{'Result':<10}")
    print("-" * 34)
    for length in test_cases:
        result = validate_segment(length)
        print(f"{'Case':<12}{length!s:<12}{result!s:<10}")
        print()
```

***Execution with Three (or more) Test Cases:***

```
Test Case   Length (m)  Result
-------------------------------------
Case        50          True

Case        100         True

Error: 120 m exceeds the IEEE 802.3 maximum of 100 m for a UTP segment.
Case        120         False

Error: -5 m is not a valid cable length (must be > 0).
Case        -5          False

Case        99.9        True
```

***Interpretation of Results:***

- 50 m and 99.9 m → True: both lengths are within the 100 m ceiling, so the segment is compliant and will support reliable Ethernet signalling without excessive attenuation.
- 100 m → True: the function treats the boundary value as valid, matching the IEEE 802.3 specification of 'up to 100 metres'.
- 120 m → False, with error message: this length exceeds the permissible limit; in practice, a segment this long would suffer signal attenuation and timing violations severe enough to cause frame errors.
- -5 m → False, with error message: a negative length is physically meaningless, so the function safely rejects it with input-validation logic rather than crashing.

Overall, the function correctly distinguishes compliant cable runs from non-compliant ones, and its explicit error messages make it easy to diagnose exactly why a given segment failed validation.

## Question 3 — CSMA/CD Simulation (4 Hosts, Star Topology)

This program simulates CSMA/CD among four hosts (H1–H4) connected to a central switch. For every frame, a host performs carrier sensing: if another host happens to transmit in the same time slot, a collision is detected, a random back-off interval is computed using binary exponential back-off (range 0 to 2^n − 1 slots, where n is the collision count for that frame), and the host waits before retrying. All attempts, collisions, and back-off intervals are written to csma_log.txt.

***Source Code (q3_csma_cd.py):***

```
"""
Q3: CSMA/CD Simulation in a 4-host Star Topology
Simulates carrier sensing, collision detection, random back-off (binary
exponential back-off, as in real Ethernet) and retransmission. All events
are written to csma_log.txt.
"""

import random

LOG_FILE = "csma_log.txt"
MAX_ATTEMPTS = 5
SLOT_TIME = 1  # arbitrary time unit


class Host:
    def __init__(self, host_id):
        self.host_id = host_id
        self.backoff_stage = 0  # n, used for 2^n - 1 slot range


def carrier_busy(transmitting_hosts):
    """Collision occurs when 2+ hosts transmit in the same slot."""
    return len(transmitting_hosts) > 1


def simulate_csma_cd(hosts, num_frames, log):
    for frame_no in range(1, num_frames + 1):
        sender = random.choice(hosts)
        sender.backoff_stage = 0
        attempt = 0
        sent = False

        while attempt < MAX_ATTEMPTS and not sent:
            attempt += 1
            # Simulate whether another host also happens to transmit (collision chance)
            other_transmitting = random.random() < 0.35  # collision probability
            transmitting_now = [sender.host_id]
            if other_transmitting:
                other = random.choice([h.host_id for h in hosts if h.host_id != sender.host_id])
                transmitting_now.append(other)

            log_line = (f"Frame {frame_no} | Attempt {attempt} | "
                        f"Sender: {sender.host_id} | Carrier sense: transmitting "
                        f"hosts = {transmitting_now}")
            print(log_line)
            log.write(log_line + "\n")

            if carrier_busy(transmitting_now):
```

```
            sender.backoff_stage = min(sender.backoff_stage + 1, 10)
            max_slots = 2 ** sender.backoff_stage - 1
            backoff_slots = random.randint(0, max_slots)
            backoff_time = backoff_slots * SLOT_TIME
            collision_line = (f"  >> COLLISION detected between {transmitting_now}. "
                              f"Back-off stage={sender.backoff_stage}, "
                              f"waiting {backoff_time} time unit(s) before retry.")
            print(collision_line)
            log.write(collision_line + "\n")
        else:
            success_line = (f"  >> Frame {frame_no} transmitted SUCCESSFULLY "
                            f"by {sender.host_id} on attempt {attempt}.")
            print(success_line)
            log.write(success_line + "\n")
            sent = True

    if not sent:
        fail_line = f"Frame {frame_no} DROPPED after {MAX_ATTEMPTS} failed attempts.\n"
        print(fail_line)
        log.write(fail_line + "\n")
    else:
        log.write("\n")


def main():
    random.seed(7)  # reproducible demo run
    hosts = [Host(f"H{i}") for i in range(1, 5)]  # 4 hosts, star topology via Switch-A1

    with open(LOG_FILE, "w") as log:
        log.write("CSMA/CD Simulation Log - Star Topology (4 hosts, Switch-A1)\n")
        log.write("=" * 60 + "\n\n")
        simulate_csma_cd(hosts, num_frames=6, log=log)

    print(f"\nSimulation complete. Full event log written to '{LOG_FILE}'.")


if __name__ == "__main__":
    main()
```

***Console Output (sample run, 6 frames):***

```
Frame 1 | Attempt 1 | Sender: H3 | Carrier sense: transmitting hosts = ['H3']
  >> Frame 1 transmitted SUCCESSFULLY by H3 on attempt 1.
Frame 2 | Attempt 1 | Sender: H4 | Carrier sense: transmitting hosts = ['H4']
  >> Frame 2 transmitted SUCCESSFULLY by H4 on attempt 1.
Frame 3 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 3 transmitted SUCCESSFULLY by H1 on attempt 1.
Frame 4 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 4 transmitted SUCCESSFULLY by H1 on attempt 1.
Frame 5 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 5 transmitted SUCCESSFULLY by H1 on attempt 1.
Frame 6 | Attempt 1 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H3']
  >> COLLISION detected between ['H2', 'H3']. Back-off stage=1, waiting 1 time unit(s) before retry.
Frame 6 | Attempt 2 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H1']
  >> COLLISION detected between ['H2', 'H1']. Back-off stage=2, waiting 3 time unit(s) before retry.
Frame 6 | Attempt 3 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H4']
  >> COLLISION detected between ['H2', 'H4']. Back-off stage=3, waiting 1 time unit(s) before retry.
Frame 6 | Attempt 4 | Sender: H2 | Carrier sense: transmitting hosts = ['H2']
  >> Frame 6 transmitted SUCCESSFULLY by H2 on attempt 4.
```

```
Simulation complete. Full event log written to 'csma_log.txt'.
```

***Contents of csma_log.txt (generated log file):***

```
CSMA/CD Simulation Log - Star Topology (4 hosts, Switch-A1)
============================================================

Frame 1 | Attempt 1 | Sender: H3 | Carrier sense: transmitting hosts = ['H3']
  >> Frame 1 transmitted SUCCESSFULLY by H3 on attempt 1.

Frame 2 | Attempt 1 | Sender: H4 | Carrier sense: transmitting hosts = ['H4']
  >> Frame 2 transmitted SUCCESSFULLY by H4 on attempt 1.

Frame 3 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 3 transmitted SUCCESSFULLY by H1 on attempt 1.

Frame 4 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 4 transmitted SUCCESSFULLY by H1 on attempt 1.

Frame 5 | Attempt 1 | Sender: H1 | Carrier sense: transmitting hosts = ['H1']
  >> Frame 5 transmitted SUCCESSFULLY by H1 on attempt 1.

Frame 6 | Attempt 1 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H3']
  >> COLLISION detected between ['H2', 'H3']. Back-off stage=1, waiting 1 time unit(s) before retry.
Frame 6 | Attempt 2 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H1']
  >> COLLISION detected between ['H2', 'H1']. Back-off stage=2, waiting 3 time unit(s) before retry.
Frame 6 | Attempt 3 | Sender: H2 | Carrier sense: transmitting hosts = ['H2', 'H4']
  >> COLLISION detected between ['H2', 'H4']. Back-off stage=3, waiting 1 time unit(s) before retry.
Frame 6 | Attempt 4 | Sender: H2 | Carrier sense: transmitting hosts = ['H2']
  >> Frame 6 transmitted SUCCESSFULLY by H2 on attempt 4.
```

***Analysis — How CSMA/CD Minimizes Transmission Conflicts:***

- Carrier Sense: before transmitting, every host checks whether the shared medium (as seen through the switch/hub segment) is idle, which prevents many collisions from ever starting.
- Collision Detection: if two hosts transmit at nearly the same time (as with Frame 6, H2 vs. H3/H1/H4 above), the collision is detected immediately rather than being discovered later, which saves bandwidth compared to letting a corrupted frame run to completion.
- Random Back-off (Exponential): after each collision the contending host waits a random number of slot times drawn from a doubling range (2^n − 1). This randomization means the same two hosts are unlikely to collide again immediately, and the growing range reduces network-wide contention as traffic increases.
- Bounded retries: the simulation caps attempts at 5 per frame, mirroring real Ethernet's practice of dropping a frame after excessive collisions instead of retrying forever.

Together these mechanisms let many hosts share one physical/logical channel efficiently: collisions still occur occasionally under load, but they are detected quickly and resolved through fair, randomized retransmission rather than degrading into repeated deadlock.

## Question 4 — Client-Server Simulation with MAC Address Table

A client-server application was built with Python's socket library to simulate an Ethernet switch. The server listens for TCP connections representing switch ports; each client (host) sends a frame formatted as SRC_MAC,DST_MAC,MESSAGE. On receipt, the server performs MAC learning — it records which port the source MAC arrived on — looks up the destination MAC in its table, forwards accordingly (or floods if unknown), and prints the updated MAC address table.

***Server Source Code (q4_server.py):***

```
"""
Q4 (server): Simulated Ethernet switch server.
Receives frames of the form "SRC_MAC,DST_MAC,MESSAGE" from clients connected
to a given port, learns the source MAC -> port mapping, forwards the frame,
and prints the updated MAC address table after every successful transmission.
"""

import socket
import threading

HOST = "127.0.0.1"
PORT = 65432

mac_table = {}          # mac_address -> port_number
mac_table_lock = threading.Lock()


def print_mac_table():
    print("\n--- MAC Address Table (StarSwitch) ---")
    print(f"{'MAC Address':<20}{'Port':<10}")
    print("-" * 30)
    for mac, port in mac_table.items():
        print(f"{mac:<20}{port:<10}")
    print("-" * 30)


def handle_client(conn, addr, port_id):
    with conn:
        data = conn.recv(1024).decode()
        if not data:
            return
        try:
            src  mac, dst  mac, message = data.split(",", 2)
        except ValueError:
            conn.sendall(b"ERROR: malformed frame")
            return

        with mac_table_lock:
            mac_table[src_mac] = port_id       # MAC learning
            dst_port = mac_table.get(dst_mac, "UNKNOWN (flood)")
            print(f"[Switch] Learned {src_mac} on port {port_id}")
            print(f"[Switch] Forwarding frame from {src_mac} to {dst_mac} "
                  f"-> port {dst_port}")
            print_mac_table()

        conn.sendall(f"ACK: frame forwarded to {dst_mac}".encode())
```

```
def run_server(max_clients=4):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(max_clients)
        print(f"[Switch] Listening on {HOST}:{PORT} (Star topology central switch)")

        for port_id in range(1, max_clients + 1):
            conn, addr = s.accept()
            handle_client(conn, addr, port_id)


if __name__ == "__main__":
    run_server(max_clients=4)
```

### *Client Source Code (q4_client.py):*

```
"""
Q4 (client): sends one simulated Ethernet frame "SRC_MAC,DST_MAC,MESSAGE"
to the StarSwitch server and prints the acknowledgement.
"""
import socket
import sys

HOST = "127.0.0.1"
PORT = 65432


def send_frame(src_mac, dst_mac, message):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((HOST, PORT))
        frame = f"{src_mac},{dst_mac},{message}"
        s.sendall(frame.encode())
        response = s.recv(1024).decode()
        print(f"[Client {src_mac}] Sent frame -> {frame}")
        print(f"[Client {src_mac}] Server response: {response}")


if __name__ == "__main__":
    src, dst, msg = sys.argv[1], sys.argv[2], sys.argv[3]
    send_frame(src, dst, msg)
```

### *Sample Run — Server Console Output (4 hosts connect in turn):*

```
[Switch] Listening on 127.0.0.1:65432 (Star topology central switch)
[Switch] Learned AA:AA:AA:AA:AA:01 on port 1
[Switch] Forwarding frame from AA:AA:AA:AA:AA:01 to AA:AA:AA:AA:AA:02 -> port UNKNOWN (flood)

--- MAC Address Table (StarSwitch) ---
MAC Address        Port
------------------------------
AA:AA:AA:AA:AA:01   1
------------------------------
[Switch] Learned AA:AA:AA:AA:AA:02 on port 2
[Switch] Forwarding frame from AA:AA:AA:AA:AA:02 to AA:AA:AA:AA:AA:01 -> port 1

--- MAC Address Table (StarSwitch) ---
MAC Address        Port
```

```
------------------------------
AA:AA:AA:AA:AA:01   1
AA:AA:AA:AA:AA:02   2
------------------------------
[Switch] Learned AA:AA:AA:AA:AA:03 on port 3
[Switch] Forwarding frame from AA:AA:AA:AA:AA:03 to AA:AA:AA:AA:AA:01 -> port 1

--- MAC Address Table (StarSwitch) ---
MAC Address        Port
------------------------------
AA:AA:AA:AA:AA:01   1
AA:AA:AA:AA:AA:02   2
AA:AA:AA:AA:AA:03   3
------------------------------
[Switch] Learned AA:AA:AA:AA:AA:04 on port 4
[Switch] Forwarding frame from AA:AA:AA:AA:AA:04 to AA:AA:AA:AA:AA:02 -> port 2

--- MAC Address Table (StarSwitch) ---
MAC Address        Port
------------------------------
AA:AA:AA:AA:AA:01   1
AA:AA:AA:AA:AA:02   2
AA:AA:AA:AA:AA:03   3
AA:AA:AA:AA:AA:04   4
------------------------------
```

***Client Console Output:***

```
[Client AA:AA:AA:AA:AA:01] Sent frame -> AA:AA:AA:AA:AA:01,AA:AA:AA:AA:AA:02,Hello from H1
[Client AA:AA:AA:AA:AA:01] Server response: ACK: frame forwarded to AA:AA:AA:AA:AA:02
[Client AA:AA:AA:AA:AA:02] Sent frame -> AA:AA:AA:AA:AA:02,AA:AA:AA:AA:AA:01,Hello back from H2
[Client AA:AA:AA:AA:AA:02] Server response: ACK: frame forwarded to AA:AA:AA:AA:AA:01
[Client AA:AA:AA:AA:AA:03] Sent frame -> AA:AA:AA:AA:AA:03,AA:AA:AA:AA:AA:01,Hi from H3
[Client AA:AA:AA:AA:AA:03] Server response: ACK: frame forwarded to AA:AA:AA:AA:AA:01
[Client AA:AA:AA:AA:AA:04] Sent frame -> AA:AA:AA:AA:AA:04,AA:AA:AA:AA:AA:02,Hi from H4
[Client AA:AA:AA:AA:AA:04] Server response: ACK: frame forwarded to AA:AA:AA:AA:AA:02
```

***Explanation — MAC Learning and Frame Forwarding in Ethernet Switches:***

- MAC Learning: every time a frame arrives, the switch inspects the source MAC address and associates it with the incoming port in its MAC address table (as seen when H1, H2, H3, and H4 are each learned on ports 1–4 in turn).
- Frame Forwarding / Filtering: the switch inspects the destination MAC address and looks it up in the table. If found (a 'hit'), the frame is forwarded only to that specific port, exactly as shown when H2's frame to H1 is sent straight to port 1.
- Flooding: if the destination MAC is not yet known (as with H1's very first frame, before anyone else had been learned), the switch floods the frame out of every port except the source, ensuring delivery while it continues to learn the network.
- Aging: real switches also age out MAC table entries after a period of inactivity so that the table stays accurate as hosts move or disconnect — this is the counterpart to remove_port() used in the StarSwitch class in Question 5.

## Question 5 — StarSwitch Class (Ethernet Switch Model)

The StarSwitch class models an Ethernet switch's essential operations. add_port(host_id) connects a new host and assigns it the next free port number. remove_port(host_id) disconnects a host and frees its port. broadcast(source_host, frame) delivers a frame to every currently connected host except the source, and every such broadcast is recorded in an internal log with the source host and the list of hosts that received the frame.

***Source Code (q5_star_switch.py):***

```
"""
Q5: StarSwitch class - models basic Ethernet switch functionality
(add_port, remove_port, broadcast) with a log of every broadcast operation.
"""


class StarSwitch:
    def __init__(self, name="Switch-A1"):
        self.name = name
        self.ports = {}      # host_id -> port number
        self._next_port = 1
        self.broadcast_log = []

    def add_port(self, host_id):
        if host_id in self.ports:
            print(f"[{self.name}] Host '{host_id}' is already connected on "
                  f"port {self.ports[host_id]}.")
            return False
        self.ports[host_id] = self._next_port
        print(f"[{self.name}] Connected host '{host_id}' on port {self._next_port}.")
        self._next_port += 1
        return True

    def remove_port(self, host_id):
        if host_id not in self.ports:
            print(f"[{self.name}] Cannot remove '{host_id}': not connected.")
            return False
        port = self.ports.pop(host_id)
        print(f"[{self.name}] Disconnected host '{host_id}' from port {port}.")
        return True

    def broadcast(self, source_host, frame):
        if source_host not in self.ports:
            print(f"[{self.name}] Broadcast failed: source '{source_host}' "
                  f"is not connected.")
            return []

        destinations = [h for h in self.ports if h != source_host]
        log_entry = {
            "source": source_host,
            "frame": frame,
            "destinations": destinations,
        }
        self.broadcast_log.append(log_entry)

        print(f"[{self.name}] Broadcast from '{source_host}': \"{frame}\"")
        for dest in destinations:
            print(f"    -> delivered to '{dest}' (port {self.ports[dest]})")
        return destinations
```

```
    def show_active_ports(self):
        print(f"\nActive ports on {self.name}:")
        for host, port in self.ports.items():
            print(f"  Port {port}: {host}")

    def show  broadcast  log(self):
        print(f"\nBroadcast log for {self.name}:")
        for i, entry in enumerate(self.broadcast_log, start=1):
            print(f"  {i}. {entry['source']} -> {entry['destinations']} : "
                  f"\"{entry['frame']}\"")


if __name__ == "__main__":
    switch = StarSwitch("Switch-A1")

    for host in ["H1", "H2", "H3", "H4"]:
        switch.add_port(host)

    switch.show_active_ports()

    switch.broadcast("H1", "ARP Request: Who has 192.168.1.4?")
    switch.broadcast("H3", "DHCP Discover")

    switch.remove_port("H2")
    switch.show_active_ports()

    switch.broadcast("H1", "Second ARP Request after H2 left")

    switch.show_broadcast_log()
```

***Sample Execution:***

```
[Switch-A1] Connected host 'H1' on port 1.
[Switch-A1] Connected host 'H2' on port 2.
[Switch-A1] Connected host 'H3' on port 3.
[Switch-A1] Connected host 'H4' on port 4.

Active ports on Switch-A1:
  Port 1: H1
  Port 2: H2
  Port 3: H3
  Port 4: H4
[Switch-A1] Broadcast from 'H1': "ARP Request: Who has 192.168.1.4?"
    -> delivered to 'H2' (port 2)
    -> delivered to 'H3' (port 3)
    -> delivered to 'H4' (port 4)
[Switch-A1] Broadcast from 'H3': "DHCP Discover"
    -> delivered to 'H1' (port 1)
    -> delivered to 'H2' (port 2)
    -> delivered to 'H4' (port 4)
[Switch-A1] Disconnected host 'H2' from port 2.

Active ports on Switch-A1:
  Port 1: H1
  Port 3: H3
  Port 4: H4
[Switch-A1] Broadcast from 'H1': "Second ARP Request after H2 left"
    -> delivered to 'H3' (port 3)
```

```
   -> delivered to 'H4' (port 4)

Broadcast log for Switch-A1:
  1. H1 -> ['H2', 'H3', 'H4'] : "ARP Request: Who has 192.168.1.4?"
  2. H3 -> ['H1', 'H2', 'H4'] : "DHCP Discover"
  3. H1 -> ['H3', 'H4'] : "Second ARP Request after H2 left"
```

***Demonstration Summary and Explanation:***

Four hosts (H1–H4) are added to Switch-A1, each receiving a distinct port. Two broadcasts are then sent — one from H1 and one from H3 — and in both cases the frame reaches every other active host but never loops back to its own sender, which matches real switch behaviour (a switch never forwards a frame back out the port it arrived on). After H2 is disconnected with remove_port(), a third broadcast from H1 correctly reaches only the two hosts that remain connected (H3 and H4), confirming that the active-port record is updated immediately.

Significance of broadcasting in Ethernet communication:

- Address resolution: protocols such as ARP rely on broadcast frames to ask 'who has this IP address?' when the corresponding MAC address is not yet known.
- Service discovery: DHCP clients broadcast a Discover message because they do not yet have an IP address (or the server's address) to send a unicast request to.
- Unknown-destination flooding: as in Question 4, a switch broadcasts (floods) a frame whenever the destination MAC is not present in its address table, guaranteeing delivery even before the table is fully populated.
- Trade-off: broadcasting is essential for these bootstrap-type communications, but excessive broadcast traffic ('broadcast storms') wastes bandwidth, which is why switches maintain per-host MAC tables to forward unicast traffic directly rather than broadcasting every frame.